

could keep track of what the user does in his browser. This is important to make sure that the server side of the application does not send any invalid or unnecessary data to the client. Qcubed, using its QForm library, however, takes a snapshot of the page and saves it on the server side before sending the HTML to the client. This state is used in later requests from the client. This technique improves the application reliability, and reduces the programmer's work and complexity of design by a very large margin.

What is QForm?

In the previous article, we have already looked at Codegen, which helps build forms, and takes care of the 'Model' part of Qcubed's MVC architecture. QForm is the part of Qcubed that handles the 'View' and 'Controller' parts of MVC. QForm is responsible for creation of the page, along with elements (we would call them controls) on the page (text-boxes, buttons, text, etc.) and helps the developer determine the events that will trigger actions, and the way in which those actions are to be handled. Before we go into code, let us first answer an interesting question: "If QForm handles the Web page, why is it named QForm, not QPage?"

The reason is, Qcubed treats the entire page as a 'Form' that submits to itself. This technique makes sure that the client does not have to know the names of other files used in the process, nor their paths, nor their function names—a better level of security, you see! For example, if you allow the user to insert data in *newpost.php*, while *insert_newpost.php* handles the values from the form submission and inserts data into the database, then a bug in *insert_newpost.php* can possibly open a plethora of vulnerabilities! Qcubed would normally hide the information about which file handles which request on the server. This technique has another benefit; it helps handle multiple events and actions easily. Now, without further ado, we will delve into the code.

Building the first Web page in Qcubed

In previous articles, we created a small MySQL database to store blog posts and comments. Let us create a Web page that allows us to insert a new blog post in the database.

Code

Create a file named *newpost.php* in the root directory of your Qcubed installation, and put the following code in it:

```
<?php
require_once 'qcubed.inc.php';

class NewPost extends QForm {
    // Declaring elements-Input Boxes
    protected $txtTitle, $txtBody;
    // Submit button
```

```
protected $btnSubmit;
// Text labels
protected $lblMessage, $lblResult;
protected function Form_Create() {
    // Set message at top
    $this->lblMessage = new QLabel($this);
    $this->lblMessage->Text = "<strong>Create new Blog
Post</strong>";
    // Define Title input box
    $this->txtTitle = new QTextBox($this);
    // Define Body input box
    $this->txtBody = new QTextBox($this);
    // make it a textarea
    $this->txtBody->TextMode = QTextMode::MultiLine;
    // Submit Button
    $this->btnSubmit = new QButton($this);
    $this->btnSubmit->Text = 'Submit';
    $this->btnSubmit->AddAction(new QClickEvent(), new
QAjaxAction('btnSubmit_Click'));
    // Result text
    $this->lblResult = new QLabel($this);
}

protected function btnSubmit_Click($strFormId,
$strCtrlId, $strParameter) {
    // Create new post
    $objNewPost = new Post();
    // Load defaults
    $objNewPost->Initialize();
    // Assign values from the form
    $objNewPost->Title = $this->txtTitle->Text;
    $objNewPost->Body = $this->txtBody->Text;
    // Save
    $objNewPost->Save();
    // Saved. Show the result
    $this->lblResult->Text = "Post was saved";
    // Empty the textboxes
    $this->txtTitle->Text = '';
    $this->txtBody->Text = '';
}
}

NewPost::Run('NewPost');
```

And in the same directory, create another file named *newpost.tpl.php*, and put the following in it:

```
<html><head>
<title>Create new Blog Post</title>
<link rel="stylesheet" type="text/css" href="<?php echo __
CSS_ASSETS__ . '/styles.css" ?">"/ >
</head><body>
<?php $this->RenderBegin(); ?>
```